QuestBoard â€” Project Report

Mobile Application Development Assignment

Table of Contents

1. Activity 1 â€” Planning, Case Scenario & Contract
2. Activity 2 â€” Prototyping, Specification, Architecture & Design
3. Activity 3 â€” Implementation
4. Activity 4 â€” Video Demo
5. Submission Checklist

Activity 1 â€” Planning, Case Scenario & Contract

1.1 Background

QuestBoard is an offline-first, gamified personal productivity application developed for individual end-users who struggle to sustain motivation across daily tasks and long-term goals. The target market spans university students, young professionals, and self-improvement enthusiasts aged 18â€“30 who are familiar with gaming culture and respond positively to reward loops, progress metrics, and competitive self-benchmarking.

The core premise is grounded in a well-documented behavioural psychology phenomenon: the "game effect" â€” the observation that individuals who apply game mechanics (points, levels, achievements, penalties) to real-world tasks exhibit measurably higher task completion rates and sustained engagement over time. Rather than building another to-do list that fades from use within a week, QuestBoard wraps every task in the language and feedback loops of a role-playing game. Every action carries consequences: completing a task grants EXP and builds streaks; missing a deadline costs HP; sustained effort unlocks achievements and raises the player's rank.

The "company" context is the developer's own self-improvement practice â€” a personal productivity system where the developer is simultaneously the client and the user. This makes the scenario highly authentic: requirements were derived from real frustrations with existing productivity tools (Notion, Todoist, Habitica) and a desire for a darker, more immersive aesthetic not present in any major competitor.

1.2 Principal Operations & Structure

QuestBoard operates entirely offline with all data persisted locally on the device. There is no cloud backend. The application manages:

- **User authentication** â€" email + password registration and login, with bcrypt-hashed passwords stored in a local SQLite database and session tokens held in the OS secure storage keychain.
- **Quest management** â€" full CRUD lifecycle for tasks ("quests"), categorised by domain (Academic, Fitness, Life) and difficulty (Easy, Medium, Hard).
- **Game mechanics engine** â€" EXP accumulation, level-up logic, HP penalty system, daily streak tracking, streak multiplier calculation, and class rank computation.
- **Achievements system** â€" 15 predefined achievement definitions evaluated against live player statistics; newly unlocked achievements trigger in-app popups.
- **Push notifications** â€" scheduled deadline reminders (5 minutes before deadline) and immediate quest-acceptance confirmations delivered via the device notification system.
- **Camera integration** â€" photo proof of completion is captured in-app and stored locally as evidence of quest fulfilment.
- **Haptic feedback** â€" distinct vibration patterns for level-up events and HP damage.

1.3 Functional Requirements

#	Requirement
FR-01	The app shall allow a user to register with an email and password
FR-02	The app shall allow a registered user to log in and out persistently
FR-03	The app shall allow the user to create a quest with a title, description, category, difficulty, and deadline

FR-04	The app shall reject quest creation if the deadline is in the past
FR-05	The app shall award EXP upon quest completion, scaled by difficulty and daily streak multiplier
FR-06	The app shall deduct 20 HP per missed deadline, checked on startup and every minute while the app is open
FR-07	The app shall allow the user to capture a photo as proof of quest completion
FR-08	The app shall allow the user to abandon (delete) a pending quest with confirmation
FR-09	The app shall compute and display the player's class rank based on total completed quests
FR-10	The app shall display real-time rank progression (quests required for the next class)
FR-11	The app shall unlock achievements when predefined conditions are met and notify the user
FR-12	The app shall send a push notification at quest acceptance and 5 minutes before deadline
FR-13	The app shall persist all data locally with no internet connection required

FR-14	The app shall support a dark mode and a light "parchment" mode, user-switchable

1.4 Local Resources / Device Features Used

Feature	Implementation
Camera	`camera` package â€” photo proof on quest completion
Push Notifications	`flutter_local_notifications` â€” scheduled + immediate
Haptic / Vibration	`vibration` package â€— level-up & damage patterns
Offline Storage	`sqflite` â€— all data persisted in on-device SQLite
Secure Storage	`flutter_secure_storage` â€— OS keychain for session tokens
Background Task	`Timer.periodic` â€— 1-minute deadline check while app is in foreground
Splash Screen	Custom `Bootloader` widget with animated loading state

Activity 2 â€— Prototyping, Specification, Architecture & Design

2.1 UI Design Brainstorm Process

Before settling on a final aesthetic, four distinct design directions were explored and evaluated against the app's target audience, purpose, and emotional tone.

Concepts Considered

Concept	Description	Verdict
Minimalist Productivity	Clean white/grey UI, card-based tasks, neutral sans-serif fonts. Inspired by Notion and Todoist.	â€Œ Rejected â€” too generic; fails to differentiate or engage the target demographic
Corporate Dashboard	Data-heavy interface, formal blue/grey palette, KPI-style metrics.	â€Œ Rejected â€” misaligned with personal use; feels cold and uninspiring
Casual Mobile Game	Bright neon colours, cartoon icons, simple point-accumulation. Inspired by Habitica.	âš¡ Partially adopted â€” the gamification structure was borrowed; the visual style was not
Dark Fantasy RPG	Deep navy backgrounds, gold accents, cinematic typography, immersive world-building language. Inspired by *Solo Leveling* and dark RPG aesthetics.	âœ… **Selected** â€” fits the immersive, high-stakes emotional tone the app targets

Key Design Decisions

1. **Theme â€” Dark Fantasy RPG**: Chosen to create emotional investment. The language of the UI (quests, HP, rank, glory) raises the perceived stakes of everyday tasks.

2. **Typography system** â€” `Cinzel` (Google Fonts) for display and heading text to evoke medieval grandeur; `Rajdhani` for body and labels for a clean sci-fi readability; `Exo2` for numeric statistics for precision and legibility.
3. **Colour system** â€” A curated three-colour primary palette:
    - `#07071A` â€” deep navy background (dark mode)
    - `#F5C518` â€” gold accent (primary actions, EXP, rank)
    - `#E53935` â€” crimson danger (HP, failures, critical alerts)
4. **Dual theme support** â€” Default dark mode with a "Parchment" light mode (warm beige/stone tones), toggled by the user. Both modes share the same colour token system via `AppThemeExtension`.
5. **Glassmorphism** â€” AppBars and overlaid containers use `BackdropFilter` blur with semi-transparent fills for a modern, layered visual depth.

Final Design System

Defined in `lib/theme/app_theme.dart`:

Class	Role
`AppTheme`	Static `ThemeData` builders for dark and light themes
`AppThemeExtension`	7 semantic colour tokens (`bg`, `bgCard`, `bgSurface`, `textPrimary`, `textSecondary`, `textMuted`, `borderSubtle`)
`AppColors`	Static constant colours + `of(context)` theme-extension accessor
`AppGradients`	Reusable `LinearGradient` presets (`gold`, `bg`, `header`)
`AppText`	Static `TextStyle` factory methods (`display`, `heading`, `label`, `body`, `stat`)

2.2 Application Architecture

QuestBoard follows a **layered MVVM-influenced architecture** using Flutter's Provider package for reactive state management.

```
â”Œâ”€â”€â”€â
```

2.3 High-Level Communication Flow

```
graph LR
    User([User]) -->|Tap| Screen[Screen Widget]
    Screen -->|context.read| GP[GameProvider]
    GP -->|await| DB[(SQLite DB)]
    GP -->|await| Auth[AuthService]
    GP -->|await| Notif[NotificationService]
    GP -->|notifyListeners| Screen
    Auth -->|read/write| SS[SecureStorage]
    Notif -->|scheduleZoned| OS[OS Notification System]
    GP -->|Timer.periodic 1 min| GP
```

2.4 Use Case Diagram

```
graph TD
    Actor([Player])

    Actor --> UC1[Register / Login]
    Actor --> UC2[Create Quest]
    Actor --> UC3[Complete Quest]
    Actor --> UC4[Abandon Quest]
    Actor --> UC5[View Achievements]
    Actor --> UC6[View Trophy Room]
    Actor --> UC7[Toggle Theme]
    Actor --> UC8[Logout]

    UC3 --> UC3a[Capture Photo Proof]
    UC3 --> UC3b[Gain EXP + Level Up]
    UC3 --> UC3c[Update Streak]
    UC3 --> UC3d[Check Achievements]

    UC2 --> UC2a[Set Deadline]
    UC2 --> UC2b[Choose Category + Difficulty]

    UC1 --> UC1a[Onboarding â€” Set Player Name]
```

2.5 Entity Relationship Diagram

```
erDiagram
    USERS {
        TEXT id PK
        TEXT email
        TEXT password
        TEXT created_at
    }

    PLAYER_STATS {
        TEXT user_id PK
        TEXT name
        INT level
        INT current_exp
        INT hp
        INT total_completed_quests
        TEXT rank
```

```
        INT current_streak
        INT longest_streak
        TEXT last_quest_date
    }

    QUESTS {
        TEXT id PK
        TEXT user_id FK
        TEXT title
        TEXT description
        TEXT category
        TEXT difficulty
        INT exp_reward
        TEXT status
        TEXT media_proof_path
        TEXT deadline
    }

    ACHIEVEMENTS {
        TEXT user_id FK
        TEXT id
        TEXT unlocked_at
    }

    USERS ||--|| PLAYER_STATS : "has"
    USERS ||--o{ QUESTS : "owns"
    USERS ||--o{ ACHIEVEMENTS : "unlocks"
```

2.6 User Navigation Flow

```
graph TD
    A[App Launch] --> B{isLoading?}
    B -- Yes --> C[Boot Splash â€” Forging destiny...]
    B -- No --> D{isAuthenticated?}
    D -- No --> E{isFirstRun?}
    E -- Yes --> F[IntroScreen]
    E -- No --> G[AuthScreen]
    D -- Yes --> H{player == null?}
    H -- Yes --> I[OnboardingScreen]
    H -- No --> J[DashboardScreen]
    F --> G
    G --> J
    I --> J
    J --> K[CreateQuestScreen]
    J --> L[QuestCompletionScreen]
    J --> M[AchievementsScreen]
    J --> N[TrophyRoomScreen]
```

2.7 Game Mechanics Specification

EXP & Levelling

- Difficulty â†’ EXP: Easy = 20 EXP, Medium = 50 EXP, Hard = 100 EXP
- Applied EXP = `baseEXP Ã— streakMultiplier`

- Level threshold: `level Ã— 100` EXP (level 1 needs 100, level 2 needs 200, etc.)
- On level-up: HP fully restored to 100

Streak Multiplier

Streak Duration	Multiplier
0â€“2 days	1.0Ã—
3â€“6 days	1.25Ã—
7â€“13 days	1.5Ã—
14â€“29 days	1.75Ã—
30+ days	2.0Ã—

Rank System (Milestone-Based)

Completed Quests	Rank
0 â€“ 99	E-Class
100 â€“ 299	D-Class
300 â€“ 599	C-Class
600 â€“ 999	B-Class

1000 â€“ 1499	A-Class
1500+	S-Class
HP = 0 (any)	E-Class (override)

HP Penalty

- Starting HP: 100
- Missed deadline: âˆ’20 HP per quest
- Floored at 0
- Damage checked: on app startup + every 1 minute via `Timer.periodic`

Achievements (15 Total)

Group	Achievement	Condition
Milestones	First Blood	1 quest completed
Milestones	Decathlon	10 quests completed
Milestones	Centurion	50 quests completed
Specialists	Scholar	5 Academic quests
Specialists	Iron Will	5 Fitness quests
Specialists	Life Hacker	5 Life quests

Streaks	Streak Starter	3-day streak
Streaks	On Fire	7-day streak
Streaks	Unstoppable	30-day streak
Levels	Rising Star	Level 5
Levels	Veteran	Level 10
Levels	Legend	Level 25
Rank	S-Class Hunter	Reach S-Class
Difficulty	Hard Mode	5 Hard quests
Perfection	Perfectionist	10 quests, 0 failures on record

Activity 3 â€” Implementation

3.1 Technology Stack

Layer	Technology	Version
Language	Dart	â‰¥ 3.10.7
Framework	Flutter	Stable channel

Target Platform	Android (primary), iOS (secondary)	Android SDK 21+
State Management	Provider (`ChangeNotifier`)	^6.1.5
Local Database	sqflite (SQLite wrapper)	^2.4.2
Authentication	BCrypt + flutter_secure_storage	^1.1.3 / ^9.2.2

3.2 Full Dependency Inventory

Package	Version	Purpose
`sqflite`	^2.4.2	Local SQLite database (all persistent data)
`path_provider`	^2.1.5	Resolves DB and file system paths
`path`	^1.9.1	Joins file path segments
`provider`	^6.1.5	Reactive state management (ChangeNotifier)
`camera`	^0.12.0+1	In-app photo proof capture
`flutter_local_notifications`	^21.0.0	Scheduled + immediate push notifications
`vibration`	^3.1.8	Haptic feedback patterns

`intl`	^0.20.2	Date/time formatting
`uuid`	^4.5.3	UUID v4 generation for entity IDs
`timezone`	^0.11.0	Timezone-aware notification scheduling
`google_fonts`	^6.2.1	Cinzel, Rajdhani, Exo2 typefaces
`bcrypt`	^1.1.3	Password hashing (offline auth)
`flutter_secure_storage`	^9.2.2	OS keychain for session token
`shared_preferences`	^2.3.2	Theme mode + first-run flag persistence
`cupertino_icons`	^1.0.8	iOS-style icon set

3.3 Project File Structure

```
lib/
â”œâ”€â”€ main.dart                    # App entry point, boot router,
Provider setup
â”œâ”€â”€ data/
â”,   â””â”€â”€ database_helper.dart     # Singleton SQLite helper (schema
v3, CRUD ops)
â”œâ”€â”€ models/
â”,   â”œâ”€â”€ quest.dart               # Quest model (toMap / fromMap /
copyWith)
â”,   â”œâ”€â”€ player.dart              # PlayerStats model
(streakMultiplier getter)
â”,   â”œâ”€â”€ achievement.dart         # Achievement model + 15 static
definitions
â”,   â└â”€â”€ user.dart                # User model (email + hashed
password)
â”œâ”€â”€ providers/
â”,   â”œâ”€â”€ game_provider.dart       # Central game state + DB
orchestration
```

```
â”,   â””â”€â”€ theme_provider.dart      # Dark/light toggle + first-run
persistence
â”œâ”€â”€ services/
â”,   â”œâ”€â”€ auth_service.dart        # Sign-up / login / logout (BCrypt
+ SecureStorage)
â”,   â”œâ”€â”€ notification_service.dart# Init, show, schedule, cancel
notifications
â”,   â└â”€â”€ vibration_service.dart   # Level-up and damage haptic
patterns
â”œâ”€â”€ screens/
â”,   â”œâ”€â”€ intro_screen.dart        # First-launch welcome / splash
â”,   â”œâ”€â”€ onboarding_screen.dart   # New player name entry
â”,   â”œâ”€â”€ auth_screen.dart         # Login + Register tabs
â”,   â”œâ”€â”€ dashboard_screen.dart    # Main HUD: stat bars, rank
progress, quest lists
â”,   â”œâ”€â”€ create_quest_screen.dart # Quest creation form with
validation
â”,   â”œâ”€â”€ quest_completion_screen.dart # Camera-based proof of
completion
â”,   â”œâ”€â”€ achievements_screen.dart  # Badge gallery with progress
banner
â”,   â└â”€â”€ trophy_room_screen.dart   # Gallery of proof photos from
completed quests
â└â”€â”€ theme/
    â└â”€â”€ app_theme.dart           # AppTheme, AppThemeExtension,
AppColors,
                                 # AppGradients, AppText
```

3.4 Key Implementation Highlights

Offline-First Architecture

All data is stored in a local SQLite database (`questboard.db`) managed by a singleton `DatabaseHelper`. There is no network dependency at runtime. The database version is 3, with schema migrations handled via `onUpgrade`.

Authentication

Users register with an email and password. The password is hashed with BCrypt (`BCrypt.hashpw`) before being stored â€" plain-text passwords never touch the database. Upon successful login, the user's UUID is written to the device's OS secure storage keychain via `flutter_secure_storage`, persisting the session across app restarts without re-authentication.

Deadline Enforcement

`GameProvider` runs two complementary deadline checks:

1. **Startup check** â€" `_checkMissedDeadlines()` is called in `loadUserData()` each time the app opens.

2. **Polling timer** â€” `Timer.periodic(Duration(minutes: 1), ...)` fires every minute while the app is in the foreground, re-checking all pending quests against `DateTime.now()`.

Missed quests are marked `Failed`, âˆ’20 HP is applied per quest, rank is recalculated, and the player record is persisted to the database.

Notification Scheduling

`flutter_local_notifications` v21 is used with named parameters throughout. Two notifications are scheduled per quest:

- **Immediate** â€” "Quest Accepted!" shown on quest creation.
- **5 minutes before deadline** â€” "Quest Reminder: expires in 5 minutes!"
- **At deadline** â€” "Quest Failed: HP Damage!" fires at the exact deadline moment.

An exact-alarm fallback is implemented: if `AndroidScheduleMode.exactAllowWhileIdle` fails (Android 13+ permission restriction), the service retries with `AndroidScheduleMode.inexactAllowWhileIdle`.

Rank Progress Bar

The dashboard's stats header includes a live rank progress indicator. The widget computes the floor/ceiling of the player's current rank band and renders a labelled progress bar (e.g., `"42 / 100 QUESTS â†’ D-Class"`). At S-Class, it displays "ðŸ†
MAX RANK ACHIEVED".

Camera Integration

The device camera list is loaded during `GameProvider.init()` (via `availableCameras()`) and stored in the provider. The `QuestCompletionScreen` reads this list from the provider via a `postFrameCallback`, initialises a `CameraController`, and captures a photo which is saved to the app's documents directory. A graceful fallback view is shown if no camera is available.

Activity 4 â€” Video Demo

4.1 Required Watermark / Evidence

Include your name, student ID, and date as a persistent overlay or introduce yourself on camera at the start. The `flutter_local_notifications` permission prompt and the camera capture flow are strong "proof of development" moments to highlight.

Submission Checklist

Item	Status
âœ… Report PDF (this document)	Ready
âœ… GitHub repository	[https://github.com/NanaKY931/QUESTBOARD](https://github.com
âœ… APK (`build/app/outputs/flutter-apk/app-debug.apk`)	Built
â¬œ Video demo â€" YouTube upload, verify public, add link	Pending
Due Date	**30 April 2026**

> [!IMPORTANT] Before submission: (1) make the GitHub repo public, (2) confirm the YouTube video is publicly accessible by sending the link to a friend to verify, (3) add both links to the final PDF version of this report.

AI Usage Disclaimer

In accordance with academic integrity guidelines, it is declared that Artificial Intelligence (AI) assistance was utilised during the development of this project. AI tools

*were predominantly employed to accelerate the **debugging** of the Flutter application, implement and refine the **frontend UI design** (including standardising the glassmorphism aesthetic and thematic tokens), and scaffold **integration testing**. The core architecture, business logic, game mechanics, and project requirements were originally conceived and driven by the developer.*